LayerZero Gasolina

Security Assessment

December 18th, 2025 — Prepared by OtterSec

Caue Obici caue@osec.io

Nicholas R. Putra nicholas@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-LGA-ADV-00 | Lack of Protocol Type Validation 6

OS-LGA-ADV-01 | Discrepancy in Onchain/Offchain Cell Parsing Logic 8

General Findings **9**

OS-LGA-SUG-00 | Optional Message Hash Validation 10

Vulnerability Rating Scale **11**

Procedure **12**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

LayerZero engaged OtterSec to perform a review of `gasolina` program. This assessment was conducted between November 20th and November 25th, 2025. For more information on our auditing methodology, refer to chapter 07.

Key Findings

We produced 3 finding throughout this audit engagement.

In particular, we identified a critical vulnerability where the protocol type is not strictly validated, enabling an attacker to supply an invalid value that bypasses the block confirmation check while still having the block confirmation (provided in the HTTP request) consumed by ULNv2/ULNv3 calldata construction (OS-LGA-ADV-00). Additionally, a mismatch between on-chain fixed-offset cell parsing and off-chain metadata-based decoding allows attackers to manipulate storage layouts so invalid on-chain events pass off-chain validation (OS-LGA-ADV-01).

We also advised to ensure the message hash is mandatorily provided as it is currently optional, rendering it possible to bypass message hash validation by omitting the message hash parameter from the API request (OS-LGA-SUG-00).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/gasolina. This audit was performed against commit 4f09e3d.

A brief description of the program is as follows:

Name	Description
gasolina	An offchain oracle that verifies a message was committed on a source blockchain and returns a signed attestation that can be trusted by destination chains for cross-chain message execution.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 3 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.

Severity	Count
CRITICAL	2
HIGH	0
MEDIUM	0
LOW	0
INFO	1

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 06.

ID	Severity	Status	Description
OS-LGA-ADV-00	CRITICAL	RESOLVED	`protocolType` is not strictly validated, enabling an attacker to supply an invalid value that bypasses the `blockConfirmation` check in `validateReadiness` while still having `blockConfirmation` consumed by `ULNv2`/`ULNv3` calldata construction.
OS-LGA-ADV-01	CRITICAL	RESOLVED	A mismatch between on-chain fixed-offset cell parsing and off-chain metadata-based decoding allows attackers to manipulate storage layouts so invalid on-chain events pass off-chain validation.

© 2025 Otter Audits LLC. All Rights Reserved.

Lack of Protocol Type Validation CRITICAL OS-LGA-ADV-00

Description

In the current implementation, `signingContext.protocolType` is user-controlled input (originating from the HTTP request), yet it is treated as a control-flow selector that determines which safety checks are applied. `ProtocolType` may essentially be `MESSAGE` or `READ`, however there is no explicit validation that checks this since typescript types are not enforced in runtime. In `validateReadiness`, only `ProtocolType.MESSAGE` explicitly triggers assertion for `signingContext.blockConfirmation`. Thus, any other value, including an invalid enumeration, implicitly falls into the `READ` validation path, which asserts block confirmations by extracting it from the time markers inside the command. Since `signingContext` may be completely controlled by an attacker, it is possible for them to ensure `protocolType` is not `MESSAGE`.

>_ *apps/gasolina/src/app/app.ts* TYPESCRIPT

```typescript
private async validateReadiness( lzSentEvent: LZSentEventV1 | LZSentEventV2, signingContext:
    ↪  SigningContext,
): Promise<void> {
    if (signingContext.protocolType === ProtocolType.MESSAGE) {
        const currentBlockConfirmations = await this.options.rpcSdkFactory
            .getSdk(lzSentEvent.lzMessageId.pathwayId.srcChainName)
            .getBlockConfirmations(lzSentEvent.onChainEvent.txHash)
        if (currentBlockConfirmations < signingContext.blockConfirmation) {
            throw new BadRequestError(
                `block confirmations not met, current block confirmation:
                    ↪  ${currentBlockConfirmations}`,
            )
        }
    } else {
        await Promise.all([
            this.options.timeMarkerValidatorSdkFactory
                .getSdk()
                .checkResolvedTimeMarkerValidity(
                    signingContext.resolvedTimestampTimeMarkers,
                ),
            this.options.lzCmdResolverSdkFactory
                .getSdk()
                .extractTimeMarkers(lzSentEvent.message)
                [..]
        ])
    }
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

After the assertion, `DVNHashCallData` is constructed based on the `ULN` version, which is derived from `lzMessageId`. However, since `lzMessageId` originates from the HTTP request, it may also be attacker-controlled.

>_ *apps/gasolina/src/app/app.ts* TYPESCRIPT

```typescript
public async signRequestV2(
    request: GasolinaApiRequestV2,
): Promise<GasolinaApiResponse> {
    [...]
    const hashCallDataBuilder =
        this.options.hashCallDataBuilders[lzMessageId.ulnSendVersion]
    [...]
}
```

This enables the attacker to specifically choose `ULNv2` or `ULNv3`, as they do not support `READ` semantics, and do not have time marker validations (time marker validations should apply only to `ULNReadV1`), instead utilizing `signingContext.blockConfirmation` while building the calldata.

This is only possible because the `hashCallDataBuilder` for `ULNv2` and `ULNv3` only throws an error if the `protocolType` is equal to `READ`:

>_ *apps/gasolina/src/app/hashCallDataBuilder/ulnV2.ts* TYPESCRIPT

```typescript
if (args.signingContext.protocolType === ProtocolType.READ) {
    throw new Error('Invalid protocol type for ULN V2')
}
```

Therefore, if an attacker provides `ProtocolType` that is not `READ` and not `MESSAGE`, the time marker verification flow is executed and the `signingContext.blockConfirmation` is the utilized variable for the `ULN` calldata. And on choosing `ULNv2` or `ULNv3`, time-marker checks trivially succeed with empty marker sets, resulting in a bypass of the `blockConfirmation` variable assertion.

Remediation

Explicitly validate all user-controllable inputs and treat them as untrusted data.

Patch

Resolved in PR#88 by validating if the protocol type sent is either

`ProtocolType.MESSAGE` or `ProtocolType.READ`.

© 2025 Otter Audits LLC. All Rights Reserved.

Discrepancy in Onchain/Offchain Cell Parsing Logic CRITICAL OS-LGA-ADV-01

Description

There is a mismatch between how TON cells are parsed on-chain versus off-chain during event validation. For TON-based LayerZero events, `getLZSentEvent` retrieves the event via the following steps:

1. The oracle retrieves the full transaction execution trace utilizing `v3_getTransactionTrace(srcTxHash)`, which includes all external and internal messages produced by the transaction.
2. It recursively traverses this trace and flattens it into a list of messages (external and internal messages).
3. The oracle filters this list to locate the message that is expected to encode the LayerZero event.
4. Once a candidate message is found, it attempts to verify that `BaseStorage::owner` matches the controller address and ensures the message emitter corresponds to the deterministically derived contract address from the initial state (data and code).

However, since the offchain class decoder is different from the onchain program, it may result in failures when validating the real contract owner offchain. Onchain, the contract accesses `BaseStorage.owner` utilizing a hardcoded bit offset (`BaseStorage::_ownerOffset = 350`).

This implies the owner field is always read from a fixed position in the cell, regardless of how the cell is structured. Offchain, however, the oracle decodes `BaseStorage` via a dynamic, metadata-driven class decoder, where field offsets are derived from serialization metadata embedded in the cell itself. Because this metadata originates from the event message, it is effectively attacker-controlled. As a result, the off-chain parser may read the owner field from a different location than the on-chain program. This difference may be exploited to bypass the `BaseStorage::owner` validation and potentially send malicious events.

Remediation

Ensure the off-chain class decoder mirrors the on-chain parsing logic so that both implementations behave identically.

Patch

Resolved in PR#88 by retrieving the `BaseStorage.owner` from the same fixed offset as the onchain program (350).

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LGA-SUG-00	Currently, `messageHash` is optional, enabling callers to skip message integrity validation.

© 2025 Otter Audits LLC. All Rights Reserved.

Optional Message Hash Validation OS-LGA-SUG-00

Description

`validateMessageHash` only enforces integrity if `messageHash` is provided. Thus, it is possible to bypass message hash validation by omitting the `messageHash` parameter from the API request. Depending on how this is used by the Oracle Client, this allows Gasolina to sign an on-chain message without cryptographically binding the request to the caller's intended message.

>_ *apps/gasolina/src/app/app.ts* TYPESCRIPT

```typescript
private validateMessageHash(
    request: GasolinaApiRequestV2,
    sentEvent: LZSentEventV1 | LZSentEventV2,
): void {
    const messageHash = hashSentEventMessageForGasolina(sentEvent)
    if (
        request.messageHash &&
        request.messageHash.toLowerCase() !== messageHash.toLowerCase()
    ) {
        throw new BadRequestError(
            `Message hash mismatch, expected: ${request.messageHash}, got: ${messageHash}`,
        )
    }
}
```

Remediation

It is recommended to ensure the `messageHash` is mandatorily provided.

© 2025 Otter Audits LLC. All Rights Reserved.

06 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

07 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.